

Week 2 — Introduction to NumPy

In this notebook we will:

- Revise NumPy fundamentals
- Understand why NumPy is important for AI
- Connect every concept to Machine Learning
- Build mini AI examples
- Practice with real exercises

By the end, you will understand how AI frameworks internally use NumPy.

1 Why NumPy?

NumPy is the foundation of:

- Pandas
- Scikit-learn
- TensorFlow
- PyTorch

All deep learning frameworks internally use matrix operations.

Why? Because $AI = \text{Linear Algebra} + \text{Statistics} + \text{Optimization}$

You can do numerical calculations using pure Python. In the beginning, you might think Python is fast but once your data gets large, you'll start to notice slow downs.

One of the main reasons you use NumPy is because it's fast. Behind the scenes, the code has been optimized to run using C. Which is another programming language, which can do things much faster than Python.

The benefit of this being behind the scenes is you don't need to know any C to take advantage of it. You can write your numerical computations in Python using NumPy and get the added speed benefits.

If your curious as to what causes this speed benefit, it's a process called vectorization. [Vectorization](#) aims to do calculations by avoiding loops as loops can create potential bottlenecks.

NumPy achieves vectorization through a process called [broadcasting](#).

```
In [1]: import numpy as np
```

```
In [2]: import time
```

```
# Python list
lst = list(range(5_000_000))
start = time.time()
lst = [x * 2 for x in lst]
print("List time:", time.time() - start)

# NumPy array
arr = np.arange(5_000_000)
start = time.time()
arr = arr * 2
print("NumPy time:", time.time() - start)
```

List time: 0.43866610527038574

NumPy time: 0.013085126876831055

NumPy is faster because:

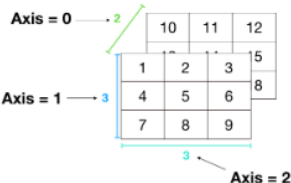
- Vectorized operations
- Implemented in C
- Optimized memory layout
- Avoids Python loops

AI models require millions of operations → speed matters.

Creating NumPy arrays

Anatomy of an array

Anatomy of a NumPy array

Data	NumPy	Details
	<code>array([1, 2, 3])</code>	• Names: Array, vector • 1-dimensional • Shape = (1, 3)
	<code>array([[1. , 2. , 3.3], [4. , 5. , 6.5]])</code>	• Names: Array, matrix • More than 1-dimension • Shape = (2, 3)
	<code>array([[[1, 2, 3], [4, 5, 6], [7, 8, 9]], [[10, 11, 12], [13, 14, 15], [16, 17, 18]])</code>	• Names: Array, matrix • More than 1-dimension • Shape = (2, 3, 3)

Key terms:

- **Array** - A list of numbers, can be multi-dimensional.
- **Scalar** - A single number (e.g. 7).

- **Vector** - A list of numbers with 1-dimension (e.g. `np.array([1, 2, 3])`).
- **Matrix** - A (usually) multi-dimensional list of numbers (e.g. `np.array([[1, 2, 3], [4, 5, 6]])`).

```
In [3]: a = np.array([1, 2, 3])
a
```

```
Out[3]: array([1, 2, 3])
```

```
In [4]: b = np.zeros((3, 3))
b
```

```
Out[4]: array([[0., 0., 0.],
               [0., 0., 0.],
               [0., 0., 0.]])
```

```
In [6]: c = np.ones((2, 4))
c
```

```
Out[6]: array([[1., 1., 1., 1.],
               [1., 1., 1., 1.]])
```

```
In [7]: d = np.random.rand(3, 3)
d
```

```
Out[7]: array([[0.11001459, 0.07765066, 0.3985114 ],
               [0.73968981, 0.31072182, 0.2204198 ],
               [0.65397863, 0.47650641, 0.52635106]])
```

Shape, Size, Dimensions

```
In [8]: matrix = np.random.rand(4, 5)
matrix
```

```
Out[8]: array([[0.29723126, 0.97495942, 0.71041755, 0.00470708, 0.2079087
7],
               [0.00525288, 0.72442576, 0.2350685 , 0.84028547, 0.2939490
4],
               [0.68387126, 0.08525329, 0.68065046, 0.15116173, 0.6528908
5],
               [0.25477666, 0.55194038, 0.13325975, 0.58674989, 0.6483800
9]])
```

```
In [9]: print("Shape:", matrix.shape)
print("Dimensions:", matrix.ndim)
print("Size:", matrix.size)
```

```
Shape: (4, 5)
```

```
Dimensions: 2
```

```
Size: 20
```

AI Interpretation:

1D → Feature vector
2D → Dataset (samples × features)
3D → Image (height × width × channels)

Example: (224, 224, 3) → RGB Image

Simulating an Image

```
In [12]: image = np.random.randint(0, 255, (224, 224, 3))  
image.shape
```

```
Out[12]: (224, 224, 3)
```

Indexing & Slicing

```
In [13]: arr = np.arange(10)  
arr
```

```
Out[13]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
In [14]: arr[2:7]
```

```
Out[14]: array([2, 3, 4, 5, 6])
```

```
In [15]: matrix = np.arange(1, 10).reshape(3, 3)  
matrix
```

```
Out[15]: array([[1, 2, 3],  
               [4, 5, 6],  
               [7, 8, 9]])
```

```
In [17]: matrix[:, 1]
```

```
Out[17]: array([2, 5, 8])
```

Broadcasting

Broadcasting allows operations between arrays of different shapes.

Very important in Neural Networks when adding bias.

Broadcasting

- What is broadcasting?
 - Broadcasting is a feature of NumPy which performs an operation across multiple dimensions of data without replicating the data. This

saves time and space. For example, if you have a 3x3 array (A) and want to add a 1x3 array (B), NumPy will add the row of (B) to every row of (A).

- Rules of Broadcasting

1. If the two arrays differ in their number of dimensions, the shape of the one with fewer dimensions is padded with ones on its leading (left) side.
2. If the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape.
3. If in any dimension the sizes disagree and neither is equal to 1, an error is raised.

The broadcasting rule: In order to broadcast, the size of the trailing axes for both arrays in an operation must be either the same size or one of them must be one.

Broadcasting

$$\begin{array}{|c|} \hline 1 \\ \hline 2 \\ \hline \end{array} * 1.6 = \begin{array}{|c|} \hline 1 \\ \hline 2 \\ \hline \end{array} * \begin{array}{|c|} \hline 1.6 \\ \hline 1.6 \\ \hline \end{array} = \begin{array}{|c|} \hline 1.6 \\ \hline 3.2 \\ \hline \end{array}$$

Basic array operations

$$\text{data} + \text{ones} = \begin{array}{|c|c|} \hline \text{data} & \\ \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} + \begin{array}{|c|c|} \hline \text{ones} & \\ \hline 1 & 1 \\ \hline 1 & 1 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 2 & 3 \\ \hline 4 & 5 \\ \hline \end{array}$$

$$\text{data} + \text{ones_row} = \begin{array}{|c|c|} \hline \text{data} & \\ \hline 1 & 2 \\ \hline 3 & 4 \\ \hline 5 & 6 \\ \hline \end{array} + \begin{array}{|c|c|} \hline \text{ones_row} & \\ \hline 1 & 1 \\ \hline \end{array} = \begin{array}{|c|c|} \hline \text{data} & \text{ones_row} \\ \hline 1 & 2 \\ \hline 3 & 4 \\ \hline 5 & 6 \\ \hline \end{array} + \begin{array}{|c|c|} \hline 1 & 1 \\ \hline 1 & 1 \\ \hline 1 & 1 \\ \hline \end{array} = \begin{array}{|c|c|} \hline 2 & 3 \\ \hline 4 & 5 \\ \hline 6 & 7 \\ \hline \end{array}$$

```
In [18]: X = np.array([[1,2,3],
                      [4,5,6]])

bias = np.array([1,1,1])
X + bias
```

```
Out[18]: array([[2, 3, 4],
               [5, 6, 7]])
```

3 Linear Algebra (Heart of AI)

Neural Networks are just matrix multiplications:

$$Y = XW + b$$

Where:

$X \rightarrow$ Input

$W \rightarrow$ Weights

$b \rightarrow$ Bias

$Y \rightarrow$ Output

```
In [19]: X = np.random.rand(5, 3)
W = np.random.rand(3, 2)

output = X @ W # or np.dot(X, W)
output
```

```
Out[19]: array([[0.74414846, 0.66539888],
               [0.8103604 , 0.8317509 ],
               [0.94362094, 0.93020456],
               [1.06203425, 1.19559977],
               [0.378237 , 0.34636129]])
```

Dot product measures similarity between vectors.

Used in:

- Recommendation systems
- NLP embeddings
- Cosine similarity

Dot product

The main two rules for dot product to remember are:

1. The **inner dimensions** must match:
 - `(3, 2) @ (3, 2)` won't work

- `(2, 3) @ (3, 2)` will work
- `(3, 2) @ (2, 3)` will work

2. The resulting matrix has the shape of the **outer dimensions**:

- `(2, 3) @ (3, 2) -> (2, 2)`
- `(3, 2) @ (2, 3) -> (3, 3)`

Note: In NumPy, `np.dot()` and `@` can be used to achieve the same result for 1-2 dimension arrays. However, their behaviour begins to differ at arrays with 3+ dimensions.

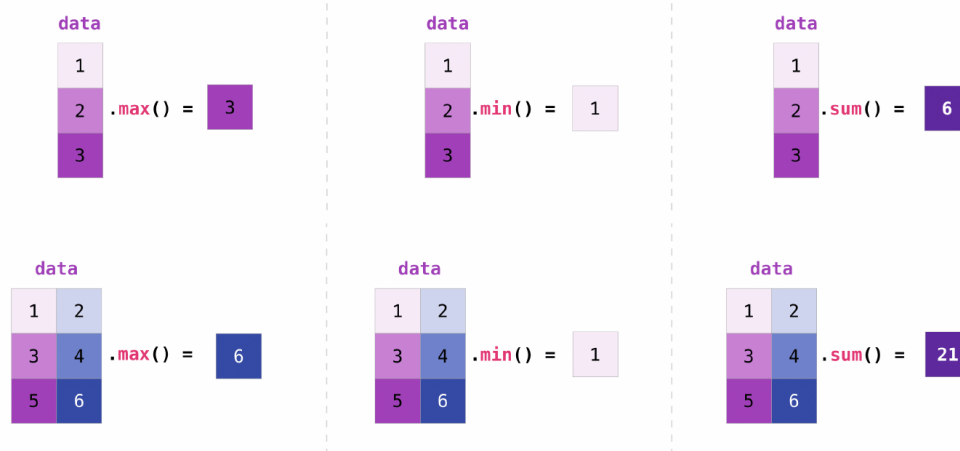
```
In [20]: v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])

np.dot(v1, v2) # or v1 @ v2
```

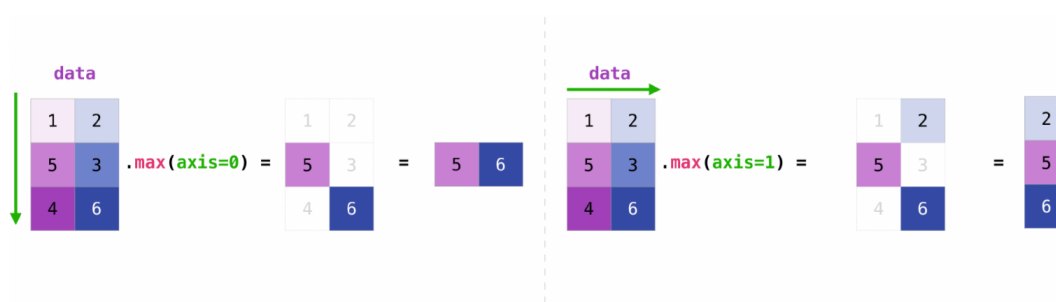
Out[20]: 32

Statistical Functions

More useful array operations



More useful array operations



```
In [21]: data = np.random.randn(1000)

print("Mean:", data.mean())
print("Std:", data.std())
print("Min:", data.min())
print("Max:", data.max())
```

```
Mean: -0.025992410531179032
Std: 0.9768297916855779
Min: -2.829690517231088
Max: 2.9845855459581543
```

Standardization

```
In [22]: X = np.random.rand(100, 3)

mean = X.mean(axis=0)
std = X.std(axis=0)

X_standardized = (X - mean) / std
X_standardized[:5]
```

```
Out[22]: array([[ -0.51667516, -0.38580176,  1.38745836],
                [-1.19974988,  1.17509901,  0.33410469],
                [-0.91233615, -0.62010639,  0.5245084 ],
                [ 0.88790179,  0.14620475,  0.94910236],
                [ 1.42981321,  0.32454948, -0.96549667]])
```

Random & Reproducibility

NumPy uses pseudo-random numbers, which means, the numbers look random but aren't really, they're predetermined.

For consistency, you might want to keep the random numbers you generate similar throughout experiments.

To do this, you can use `np.random.seed()`.

What this does is it tells NumPy, "Hey, I want you to create random numbers but keep them aligned with the seed."

Let's see it.

```
In [29]: np.random.seed(42)

X = np.random.rand(5)
X
```

```
Out[29]: array([0.37454012, 0.95071431, 0.73199394, 0.59865848, 0.15601864])
```

Transpose

A transpose reverses the order of the axes.

For example, an array with shape `(2, 3)` becomes `(3, 2)`.

```
In [33]: a = np.random.rand(5,3)
a
```

```
Out[33]: array([[0.61185289, 0.13949386, 0.29214465],
               [0.36636184, 0.45606998, 0.78517596],
               [0.19967378, 0.51423444, 0.59241457],
               [0.04645041, 0.60754485, 0.17052412],
               [0.06505159, 0.94888554, 0.96563203]])
```

```
In [34]: a.T
```

```
Out[34]: array([[0.61185289, 0.36636184, 0.19967378, 0.04645041, 0.06505159],
               [0.13949386, 0.45606998, 0.51423444, 0.60754485, 0.94888554],
               [0.29214465, 0.78517596, 0.59241457, 0.17052412, 0.96563203]])
```

Mini AI Example: Linear Regression

```
In [27]: X = np.random.rand(100, 1)
y = 3*X + 2 + np.random.randn(100, 1)*0.1

# Initialize weights
W = np.random.randn(1, 1)
b = np.random.randn(1)

# Forward pass
y_pred = X @ W + b

# Mean Squared Error
loss = np.mean((y - y_pred)**2)
loss
```

```
Out[27]: 5.585835699938655
```



Exercises

Part 1 – Fundamentals

1. Create a 5×5 matrix with random integers (0–10).
2. Extract the diagonal.
3. Replace values > 5 with 0.
4. Normalize matrix between 0 and 1.

In []:

Part 2 – Data Science

Given:

```
X = np.random.rand(200,5)
```

1. Compute mean of each column.
2. Standardize the dataset.
3. Add bias column of ones.

In []: